TOOL

Anonymous Author(s)*

ABSTRACT

1 INTRODUCTION

In recent years, the open-source software (OSS) ecosystem has witnessed substantial growth [3, 6], providing developers with a vast array of options for using free software. It has become a common practice for developers to adopt third-party OSS components to accelerate development and reduce costs [**?**]. Consequently, the vulnerability landscape has expanded beyond traditional proprietary software to encompass open-source software (OSS). The increasing vulnerabilities in open-source software have raised significant concerns, particularly regarding supply chain attacks [4]. Attackers can exploit these vulnerabilities, affecting not only the open-source software itself but also its downstream users, thus amplifying potential financial gains and cyber chaos [1].

Notably, downstream software shares or reuse logic from upstream OSS components. As these OSS components evolve and downstream software undergoes customization, the code initially derived from OSS components diverges from the original one at both ends. Consequently, it leads to challenges in mitigating *Vulnerable Code Clones* (VCCs) [**?**]. When facing incompatible changes or customized OSS components, mitigating VCCs by replacing newer OSS components becomes infeasible or cumbersome, especially in languages like C/C++ that lacks a dominant package manager.

In this context, developers often resort to manually finding and backporting security patches into downstream software. However, it is time-consuming and resource-intensive. Notably, 80% of attacks leveraged the vulnerabilities reported more than three years ago [2], indicating that the issue of VCCs has not been fully resolved.

Typically, developers employ vulnerable code clone (VCC) discovery techniques [8, 10, 13**?**] to identify reused components (such as files, functions, or fragments) in a target project program. They can also use learning-based methods for classifying vulnerable functions [5, 9**? ? ?**]. However, the effectiveness of vulnerability detection is often hindered in cases of *Vulnerabilities with Multiple Fixing Functions* (VM), where fixing patches span across multiple functions or files. Unfortunately, VM are inevitably prevalent in real world due to complex inter-procedural program logic, posing challenges for accurate VM detection.

Limitations of Existing Approaches. Existing research often addresses the detection of *Vulnerabilities with Multiple Fixing Functions* (VM) by dividing the task into matching vulnerability characteristics within individual functions (*i.e.,* matching-one-in-all approach). Clone-based techniques [8, 10, 13**?**] identify vulnerable code clones (VCCs) within functions, while learning-based approaches represent functions in abstract spaces and classify them as vulnerable or not [5, 9**? ? ?**]. However, these approaches assume equal importance for all fixing functions in a vulnerability, potentially leading to over-representation resulting in false alarms. Additionally, while software composition analysis (SCA) techniques [7, 10–12, 14] can detect VM, they are primarily designed for identifying reused OSS components, which may overlook specific vulnerability or fixing characteristics, resulting in false alarms.

Challenge. Naturally thinking, assigning all fixing equal importance is straightforward and may not accurately represent the actual characteristics of vulnerabilities. Some fixing functions may carry adequate information to represent a vulnerability, while others may not. However, it is challenging to measure and weigh the unique fingerprints of each fixing function to a vulnerability. The primary challenge lies in how to capture and distinguish the uniqueness of fixing functions and identify the critical ones for a vulnerability.

Our Approach. To tackle this challenge, we introduce TOOL, a novel approach for Vulnerabilities-with-Multiple-Fixing-Function-Detection. Unlike traditional methods that treat all fixing functions equally, TOOL prioritizes the fixing functions and selects the most critical functions for vulnerability representation (see Section **??**). It involves matching the signatures [8, 13] of these critical functions. To cope with the potential decrease in recall due to excluding the remaining fixing functions, we employ semantic equivalent statement matching, which consists of program rephrasing [**?**] (see Section **??**) and contextual semantic equivalent statement mapping (see Section **??**). The program rephrasing aims to uncover more VM by creating another signature for the rephrased form of the critical functions, while contextual semantic equivalent statement mapping aims to match the composing statements precisely based on their contextual semantic equivalence.

The insight of the contextual semantic equivalent statement mapping stems from our observation that some syntactically identical statements serve different contextual purposes. Existing approaches add new context similarity constraints using syntactically equivalent statements, which does not correct the error from the essential. We claim that "statement equivalence" should be redefined not only from syntactic similarity, but also from their contextual similarity. To our best knowledge, TOOL is the first to discriminate critical functions from fixing functions and consider both program rephrasing and contextual semantic equivalent statements to identify vulnerabilities with high quantity and quality.

Evaluation. We implemented TOOL and generated vulnerability signatures from 810 CVEs. We conducted an evaluation using a dataset comprised of the top 1,000 C/C++ real-world projects selected from GitHub. Comparing TOOL with state-of-the-art techniques, TOOL achieves a f1-score of 0.84, outperforming the best state-of-the-art at 17.6%. TOOL discovered 275 VM across 104 projects, with 42 of them confirmed by developers and 5 of them assigned with corresponding new CVE identifiers. We observed the robustness of TOOL regarding the ixing function number in the VM. We conducted an ablation study to observe the contribution of key components in TOOL. We also measured the threshold sensitivity and performance of TOOL. The results demonstrate TOOL's effectiveness and efficiency in identifying VM within real-world projects.

Contribution. We summarize our contributions as follows:

- We propose TOOL, a novel approach to detect Vulnerabilities with Multiple Fixing Functions (VM) in real-world projects.
- We evaluate TOOL on our ground truth collected from real-world C/C++ projects and demonstrate its effectiveness compared with the state-of-the-art approaches.

Table 1: Evaluated Large Language Models (LLMs)

Model	# Params	Python Fine-Tuned	Open-Source
CodeGen-350m	350 M	✓	✓
CodeGen-2b	2 B	✓	✓
CodeGen-6b	6 B	✓	✓
DeepSeek-1.3b	1.3 B	✗	✓
StarCoder2-3b	3 B	✗	✓
CodeLlama-7b	7 B	✓	✓
GPT-3.5	175 B	**UNKNOWN**	✗

- TOOL has discovered 275 vulnerabilities in 84 C/C++ projects. 42 are confirmed by developers, 5 has been assigned with CVE identifiers, demonstrating its effectiveness in real-world projects.

2 RELATED WORK

We review previous work related to

3 EMPIRICAL SETUP

In this section, we present the design of TOOL.

4 EMPIRICAL STUDY

We conduct an empirical study on three research questions.

- **RQ1. Model Perspective.**
- **RQ2. Prompt Perspective.**
- **RQ3. Library Perspective.**

4.1 Setup

5 EMPIRICAL STUDY

We conduct an empirical study on three research questions.

- **RQ1. Model Perspective.**
- **RQ2. Prompt Perspective.**
- **RQ3. Library Perspective.**

5.1 Setup

5.2 Discussion

Threats. First,

Limitations. First,

6 CONCLUSIONS

In this paper,

REFERENCES

[1] arcticwolf. 2024. *Beyond Sisense Navigating Rising Tide of Supply Chain Attacks*. Retrieved April 20, 2024 from https://arcticwolf.com/resources/blog/beyond-sisense-navigating-rising-tide-of-supply-chain-attacks/

[2] businessinsights. 2024. *10 Stats on the State of Vulnerabilities and Exploits*. Retrieved April 20, 2024 from https://www.bitdefender.com/blog/businessinsights/10-stats-on-the-state-of-vulnerabilities-and-exploits/

[3] Checkmarx. 2023. *9th Annual State of Software the Supply Chain*.

[4] crowdstrike. 2024. *Supply Chain Attacks*. Retrieved April 20, 2024 from https://www.crowdstrike.com/cybersecurity-101/cyberattacks/supply-chain-attacks/

[5] Lei Cui, Zhiyu Hao, Yang Jiao, Haiqiang Fei, and Xiaochun Yun. 2020. Vuldetector: Detecting vulnerabilities using weighted feature graph comparison. *IEEE Transactions on Information Forensics and Security* 16 (2020), 2004–2017.

[6] GitHub. 2023. *The 2023 State of the OCTOVERSE*. Retrieved April 20, 2024 from https://octoverse.github.com/

[7] Ling Jiang, Hengchen Yuan, Qiyi Tang, Sen Nie, Shi Wu, and Yuqun Zhang. 2023. Third-Party Library Dependency for Large-Scale SCA in the C/C++ Ecosystem: How Far Are We?. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 1383–1395.

[8] Seulbae Kim, Seunghoon Woo, Heejo Lee, and Hakjoo Oh. 2017. Vuddy: A scalable approach for vulnerable code clone discovery. In *2017 IEEE Symposium on Security and Privacy*. 595–614.

[9] Shu Wang, Xinda Wang, Kun Sun, Sushil Jajodia, Haining Wang, and Qi Li. 2023. GraphSPD: Graph-based security patch detection with enriched code semantics. In *2023 IEEE Symposium on Security and Privacy*. 2409–2426.

[10] Seunghoon Woo, Eunjin Choi, Heejo Lee, and Hakjoo Oh. 2023. {V1SCAN}: Discovering 1-day Vulnerabilities in Reused {C/C++} Open-source Software Components Using Code Classification Techniques. In *32nd USENIX Security Symposium*. 6541–6556.

[11] Seunghoon Woo, Sunghan Park, Seulbae Kim, Heejo Lee, and Hakjoo Oh. 2021. CENTRIS: A precise and scalable approach for identifying modified open-source software reuse. In *2021 IEEE/ACM 43rd International Conference on Software Engineering*. 860–872.

[12] Jiahui Wu, Zhengzi Xu, Wei Tang, Lyuye Zhang, Yueming Wu, Chengyue Liu, Kairan Sun, Lida Zhao, and Yang Liu. 2023. Ossfp: Precise and scalable c/c++ third-party library detection using fingerprinting functions. In *2023 IEEE/ACM 45th International Conference on Software Engineering*. 270–282.

[13] Yang Xiao, Bihuan Chen, Chendong Yu, Zhengzi Xu, Zimu Yuan, Feng Li, Binghong Liu, Yang Liu, Wei Huo, Wei Zou, et al. 2020. {MVP}: Detecting Vulnerabilities using {Patch-Enhanced} Vulnerability Signatures. In *29th USENIX Security Symposium*. 1165–1182.

[14] Can Yang, Zhengzi Xu, Hongxu Chen, Yang Liu, Xiaorui Gong, and Baoxu Liu. 2022. ModX: binary level partially imported third-party library detection via program modularization and semantic matching. In *Proceedings of the 44th International Conference on Software Engineering*. 1393–1405.